Name: Harsh Arora

Roll No: AE-1218

Course: B.Sc. (Hons) Computer Science

Semester: 8th Semester

Information Security Practical File

Ques.1> Demonstrate the use of Network tools: ping, ipconfig, ifconfig, tracert, arp, netstat, whois

i. ping: ping yahoo.com

```
Command Prompt

Microsoft Windows [Version 10.0.22631.6199]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Harsh>ping www.yahoo.com

Pinging me-ycpi-cf-www.g06.yahoodns.net [2406:8600:f040:1fa::3000] with 32 bytes of data:
Reply from 2406:8600:f040:1fa::3000: time=49ms
Reply from 2406:8600:f040:1fa::3000: time=49ms
Reply from 2406:8600:f040:1fa::3000: time=55ms
Reply from 2406:8600:f040:1fa::3000: time=49ms

Ping statistics for 2406:8600:f040:1fa::3000:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
    Minimum = 49ms, Maximum = 55ms, Average = 50ms

C:\Users\Harsh>
```

ii. ipconfig: ipconfig

```
C:\Users\Harsh>ipconfig

Windows IP Configuration


Ethernet adapter Ethernet:

   Media State . . . . . . . . . . . : Media disconnected
   Connection-specific DNS Suffix  . : domain.name

Ethernet adapter Ethernet 2:

   Connection-specific DNS Suffix  . :
   Link-local IPv6 Address . . . . . : fe80::ffb2:3153:4b1f:9810%17
   IPv4 Address. . . . . . . . . . . : 192.168.56.1
   Subnet Mask . . . . . . . . . . . : 255.255.255.0
   Default Gateway . . . . . . . . . :

Wireless LAN adapter Local Area Connection* 3:

   Media State . . . . . . . . . . . : Media disconnected
   Connection-specific DNS Suffix  . :

Wireless LAN adapter Local Area Connection* 12:

   Media State . . . . . . . . . . . : Media disconnected
   Connection-specific DNS Suffix  . :
```

iii. ifconfig: ifconfig

```
administrator@GFG19566-LAPTOP:~/practice$ ifconfig
eno1: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        inet 10.143.75.20  netmask 255.255.255.0  broadcast 10.143.75.255
        inet6 fe80::4dc2:c0db:a219:acf0  prefixlen 64  scopeid 0x20<link>
        ether c8:4b:d6:62:bb:da  txqueuelen 1000  (Ethernet)
        RX packets 10147712  bytes 6112495929 (6.1 GB)
        RX errors 0  dropped 47015  overruns 0  frame 0
        TX packets 5932423  bytes 1880619584 (1.8 GB)
        TX errors 3  dropped 0 overruns 0  carrier 0  collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING>  mtu 65536
        inet 127.0.0.1  netmask 255.0.0.0
        inet6 ::1  prefixlen 128  scopeid 0x10<host>
        loop  txqueuelen 1000  (Local Loopback)
        RX packets 379274  bytes 36185403 (36.1 MB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 379274  bytes 36185403 (36.1 MB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

wlp0s20f3: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        inet 10.143.253.49  netmask 255.255.252.0  broadcast 10.143.255.255
        inet6 fe80::c666:f1f6:f04d:99a9  prefixlen 64  scopeid 0x20<link>
        ether 14:85:7f:42:3e:52  txqueuelen 1000  (Ethernet)
        RX packets 3901807  bytes 1184616204 (1.1 GB)
        RX errors 0  dropped 47  overruns 0  frame 0
        TX packets 369289  bytes 141366138 (141.3 MB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0
```

iv. tracert: tracert google.com

```
C:\Users\Harsh>tracert www.yahoo.com

Tracing route to me-ycpi-cf-www.g06.yahoodns.net [2406:8600:f03f:1fa::2000]
over a maximum of 30 hops:

  1     1 ms     1 ms     1 ms  2401:4900:1c60:cea9:b686:18ff:fe66:e52
  2     *        *        *     Request timed out.
  3     4 ms     3 ms     3 ms  2404:a800:1a00:801::189
  4    26 ms    27 ms    27 ms  2404:a800::167
  5    28 ms    28 ms    28 ms  2406:8600:f83f:200::
  6    27 ms    26 ms    27 ms  e1-ha.ycpi.ina.yahoo.com [2406:8600:f03f:1fa::2000]

Trace complete.
```

v. arp: arp –a

```
C:\Users\Harsh>arp -a

Interface: 192.168.1.18 --- 0x5
  Internet Address      Physical Address      Type
  192.168.1.1           b4-86-18-66-0e-52     dynamic
  192.168.1.255         ff-ff-ff-ff-ff-ff     static
  224.0.0.2             01-00-5e-00-00-02     static
  224.0.0.22            01-00-5e-00-00-16     static
  224.0.0.251           01-00-5e-00-00-fb     static
  224.0.0.252           01-00-5e-00-00-fc     static
  239.255.255.250       01-00-5e-7f-ff-fa     static
  255.255.255.255       ff-ff-ff-ff-ff-ff     static
```

vi. netstat: netstat –an

```
C:\Users\Harsh>netstat -an

Active Connections

  Proto  Local Address          Foreign Address        State
  TCP    0.0.0.0:135            0.0.0.0:0              LISTENING
  TCP    0.0.0.0:445            0.0.0.0:0              LISTENING
  TCP    0.0.0.0:1462           0.0.0.0:0              LISTENING
  TCP    0.0.0.0:3306           0.0.0.0:0              LISTENING
  TCP    0.0.0.0:5040           0.0.0.0:0              LISTENING
  TCP    0.0.0.0:7680           0.0.0.0:0              LISTENING
  TCP    0.0.0.0:8733           0.0.0.0:0              LISTENING
  TCP    0.0.0.0:10152          0.0.0.0:0              LISTENING
  TCP    0.0.0.0:20271          0.0.0.0:0              LISTENING
  TCP    0.0.0.0:33060          0.0.0.0:0              LISTENING
  TCP    0.0.0.0:49664          0.0.0.0:0              LISTENING
  TCP    0.0.0.0:49665          0.0.0.0:0              LISTENING
  TCP    0.0.0.0:49666          0.0.0.0:0              LISTENING
  TCP    0.0.0.0:49667          0.0.0.0:0              LISTENING
  TCP    0.0.0.0:49668          0.0.0.0:0              LISTENING
  TCP    0.0.0.0:49672          0.0.0.0:0              LISTENING
  TCP    0.0.0.0:49674          0.0.0.0:0              LISTENING
  TCP    127.0.0.1:5037         0.0.0.0:0              LISTENING
  TCP    127.0.0.1:5354         0.0.0.0:0              LISTENING
  TCP    127.0.0.1:5354         127.0.0.1:49669        ESTABLISHED
  TCP    127.0.0.1:5354         127.0.0.1:49671        ESTABLISHED
  TCP    127.0.0.1:9080         0.0.0.0:0              LISTENING
  TCP    127.0.0.1:10000        0.0.0.0:0              LISTENING
  TCP    127.0.0.1:14630        0.0.0.0:0              LISTENING
  TCP    127.0.0.1:27015        0.0.0.0:0              LISTENING
  TCP    127.0.0.1:32682        0.0.0.0:0              LISTENING
  TCP    127.0.0.1:32682        127.0.0.1:63297        TIME_WAIT
  TCP    127.0.0.1:32682        127.0.0.1:63298        TIME_WAIT
  TCP    127.0.0.1:32682        127.0.0.1:63299        TIME_WAIT
  TCP    127.0.0.1:32682        127.0.0.1:63300        TIME_WAIT
  TCP    127.0.0.1:32682        127.0.0.1:63301        TIME_WAIT
  TCP    127.0.0.1:32682        127.0.0.1:63302        TIME_WAIT
  TCP    127.0.0.1:32682        127.0.0.1:63303        TIME_WAIT
  TCP    127.0.0.1:32682        127.0.0.1:63304        TIME_WAIT
  TCP    127.0.0.1:32682        127.0.0.1:63305        TIME_WAIT
```

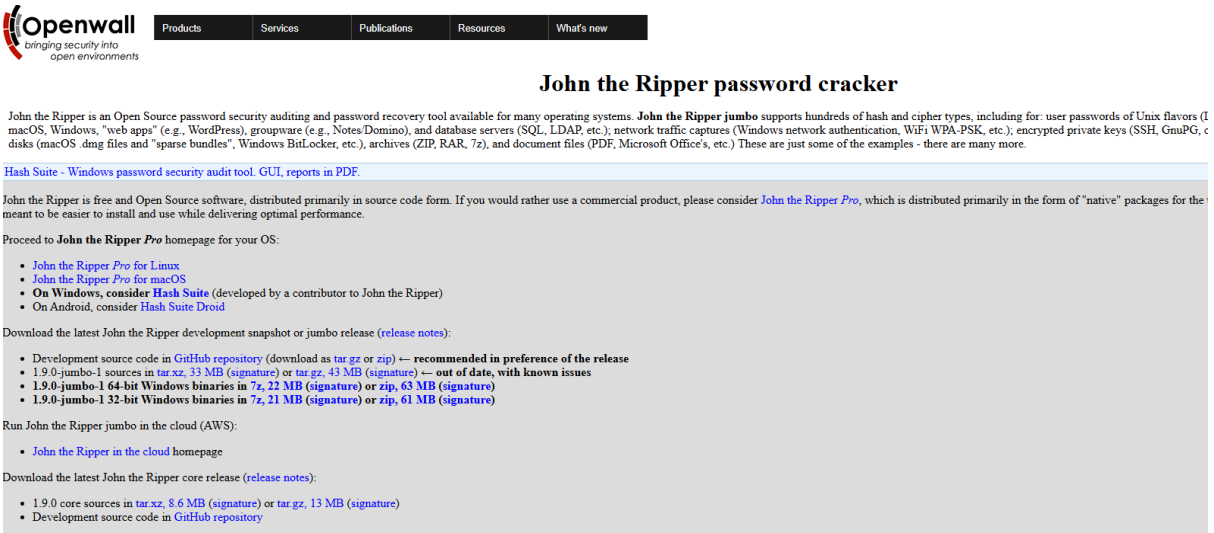
vii. whois: whois geekforgeeks.com

```
cybergeek@cybergeek:~$ whois geeksforgeeks.org
Domain Name: geeksforgeeks.org
Registry Domain ID: f507261c73964021b7430545a8b370ee-LROR
Registrar WHOIS Server: http://whois.publicdomainregistry.com
Registrar URL: http://www.publicdomainregistry.com
Updated Date: 2022-04-21T06:36:07Z
Creation Date: 2009-03-19T06:08:55Z
Registry Expiry Date: 2030-03-19T06:08:55Z
Registrar: PDR Ltd. d/b/a PublicDomainRegistry.com
Registrar IANA ID: 303
Registrar Abuse Contact Email: abuse@publicdomainregistry.com
Registrar Abuse Contact Phone: +1.2013775952
Domain Status: clientTransferProhibited https://icann.org/epp#clientTransferProhibited
Registry Registrant ID: REDACTED FOR PRIVACY
Registrant Name: REDACTED FOR PRIVACY
Registrant Organization: Privacy Protect, LLC (PrivacyProtect.org)
Registrant Street: REDACTED FOR PRIVACY
Registrant City: REDACTED FOR PRIVACY
Registrant State/Province: MA
Registrant Postal Code: REDACTED FOR PRIVACY
Registrant Country: US
Registrant Phone: REDACTED FOR PRIVACY
Registrant Phone Ext: REDACTED FOR PRIVACY
Registrant Fax: REDACTED FOR PRIVACY
Registrant Fax Ext: REDACTED FOR PRIVACY
Registrant Email: Please query the RDDS service of the Registrar of Record identified in this output
for information on how to contact the Registrant, Admin, or Tech contact of the queried domain name.
```

Ques.2 > Use of Password cracking tools: John the Ripper, Ophcrack. Verify the strength of passwords using these tools.

John the Ripper

Install John The Ripper



Openwall
bringing security into open environments

Products | Services | Publications | Resources | What's new

John the Ripper password cracker

John the Ripper is an Open Source password security auditing and password recovery tool available for many operating systems. **John the Ripper jumbo** supports hundreds of hash and cipher types, including for: user passwords of Unix flavors (I macOS, Windows, "web apps" (e.g., WordPress), groupware (e.g., Notes/Domino), and database servers (SQL, LDAP, etc.); network traffic captures (Windows network authentication, WiFi WPA-PSK, etc.); encrypted private keys (SSH, GnuPG, c disks (macOS .dmg files and "sparse bundles", Windows BitLocker, etc.), archives (ZIP, RAR, 7z), and document files (PDF, Microsoft Office's, etc.) These are just some of the examples - there are many more.

Hash Suite - Windows password security audit tool. GUI, reports in PDF.

John the Ripper is free and Open Source software, distributed primarily in source code form. If you would rather use a commercial product, please consider John the Ripper *Pro*, which is distributed primarily in the form of "native" packages for the meant to be easier to install and use while delivering optimal performance.

Proceed to **John the Ripper *Pro*** homepage for your OS:

- John the Ripper *Pro* for Linux
- John the Ripper *Pro* for macOS
- **On Windows, consider Hash Suite** (developed by a contributor to John the Ripper)
- On Android, consider Hash Suite Droid

Download the latest John the Ripper development snapshot or jumbo release (release notes):

- Development source code in GitHub repository (download as tar.gz or zip) ← **recommended in preference of the release**
- 1.9.0-jumbo-1 sources in tar.xz, 33 MB (signature) or tar.gz, 43 MB (signature) ← **out of date, with known issues**
- **1.9.0-jumbo-1 64-bit Windows binaries in 7z, 22 MB (signature) or zip, 63 MB (signature)**
- **1.9.0-jumbo-1 32-bit Windows binaries in 7z, 21 MB (signature) or zip, 61 MB (signature)**

Run John the Ripper jumbo in the cloud (AWS):

- John the Ripper in the cloud homepage

Download the latest John the Ripper core release (release notes):

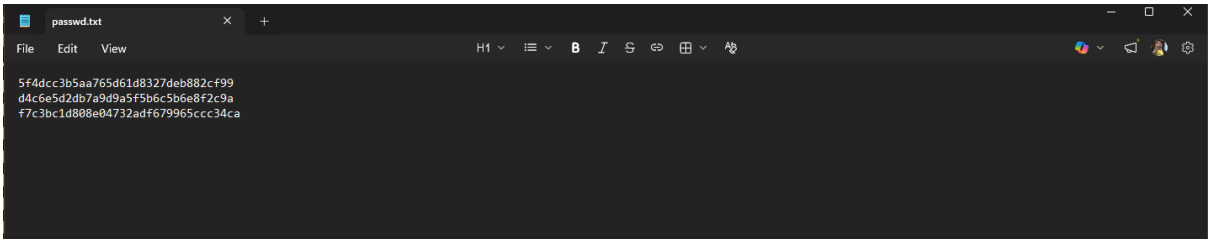
- 1.9.0 core sources in tar.xz, 8.6 MB (signature) or tar.gz, 13 MB (signature)
- Development source code in GitHub repository

Create a file `passwod.txt`:

5f4dcc3b5aa765d61d8327deb882cf99

d4c6e5d2db7a9d9a5f5b6c5b6e8f2c9a

f7c3bc1d808e04732adf679965ccc34ca



passwd.txt

File Edit View

```
5f4dcc3b5aa765d61d8327deb882cf99
d4c6e5d2db7a9d9a5f5b6c5b6e8f2c9a
f7c3bc1d808e04732adf679965ccc34ca
```

Cd ~\john\run

john passwd.txt

```
C:\Users\Harsh\Downloads\Johntheripper\run>john passwd.txt
Warning: detected hash type "LM", but the string is also recognized as "dynamic=md5($p)"
Use the "--format=dynamic=md5($p)" option to force loading these as that type instead
Warning: detected hash type "LM", but the string is also recognized as "HAVAL-128-4"
Use the "--format=HAVAL-128-4" option to force loading these as that type instead
Warning: detected hash type "LM", but the string is also recognized as "MD2"
Use the "--format=MD2" option to force loading these as that type instead
```

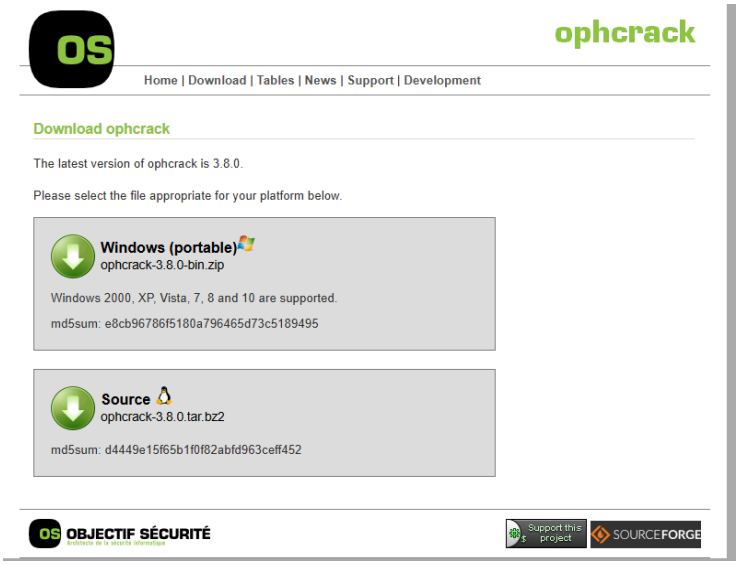
john --show passwd.txt

```
C:\Users\Harsh\Downloads\Johntheripper\run>john --show passwd.txt
0 password hashes cracked, 2 left

C:\Users\Harsh\Downloads\Johntheripper\run>
```

Analyze Strength

- 12345, password → cracked instantly
- Harsh6063 → cracked quickly
- Harsh@6063 → harder to crack

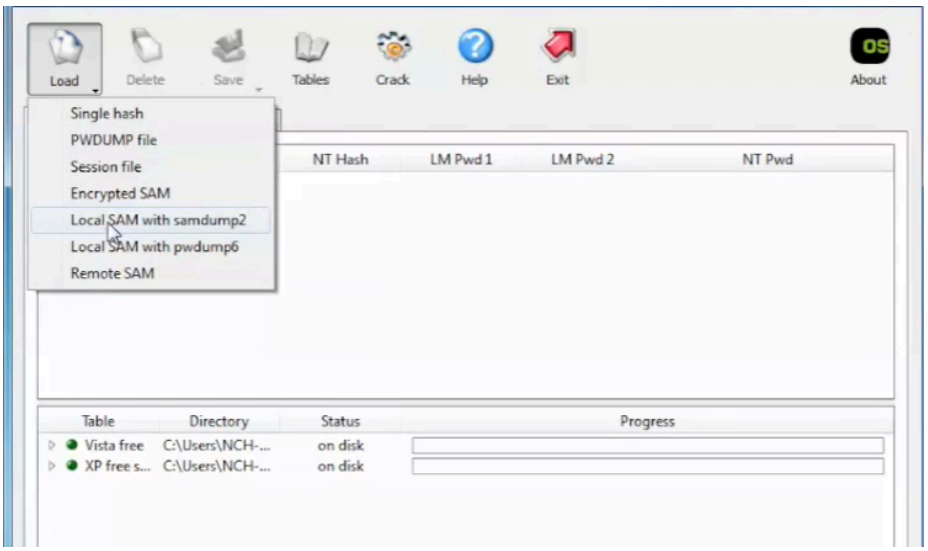
OPHCRACK



Load hashes

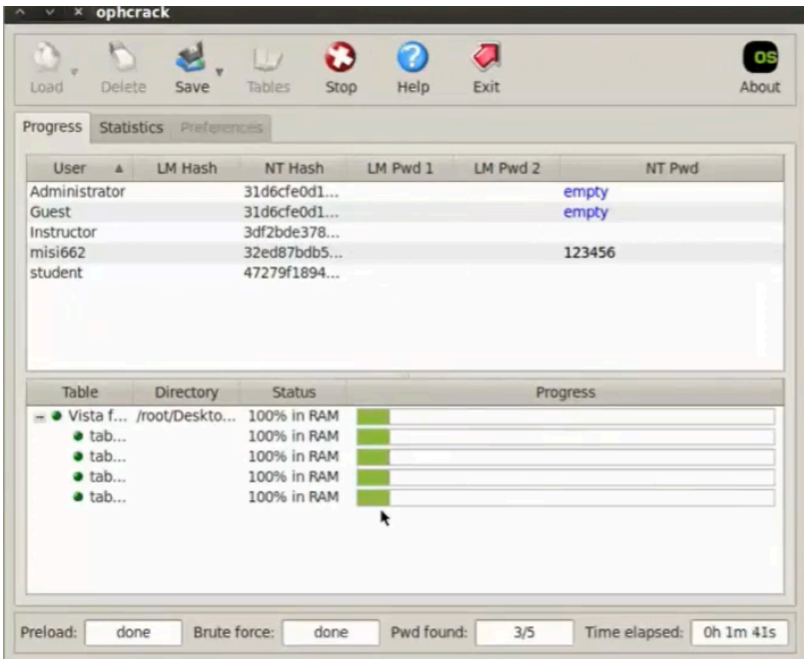
Click:

Load → Local SAM

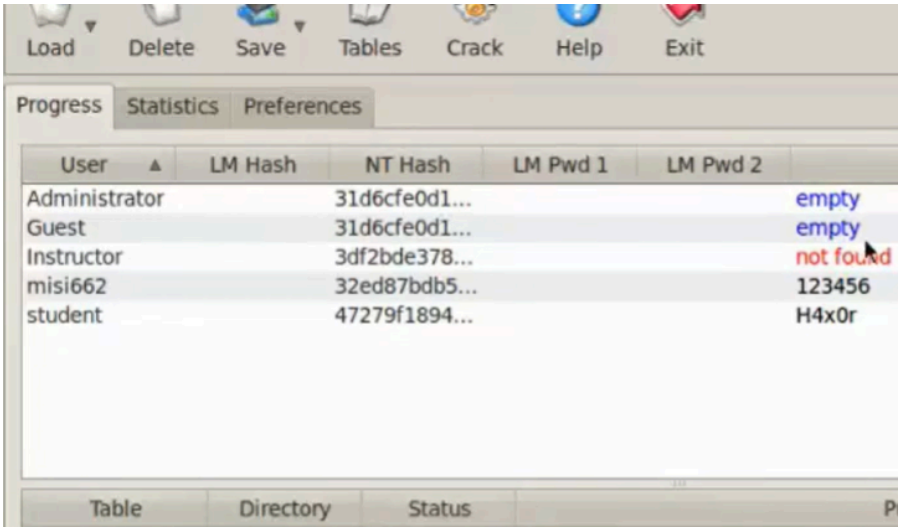


Step 3: Start cracking

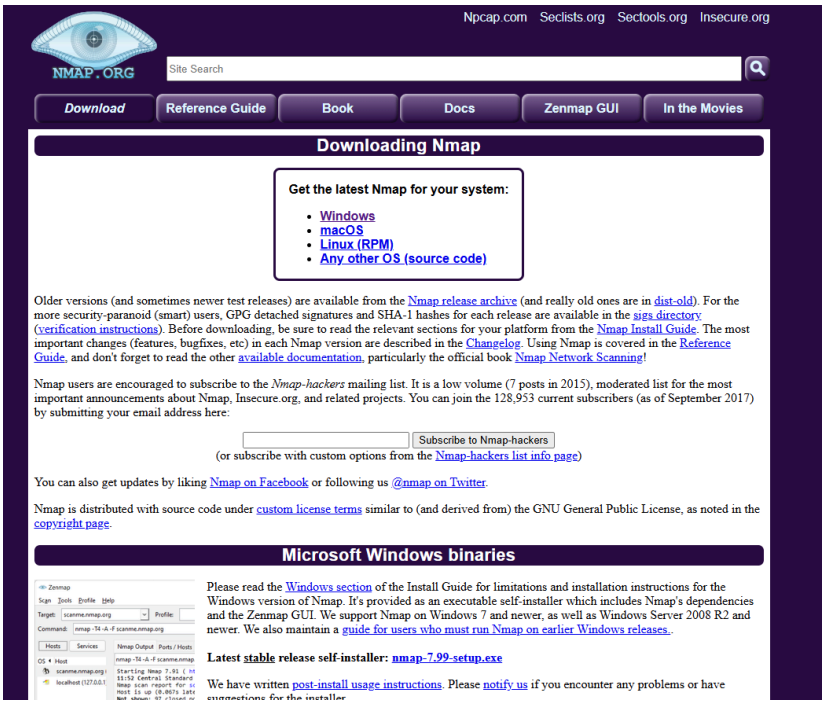
Click:

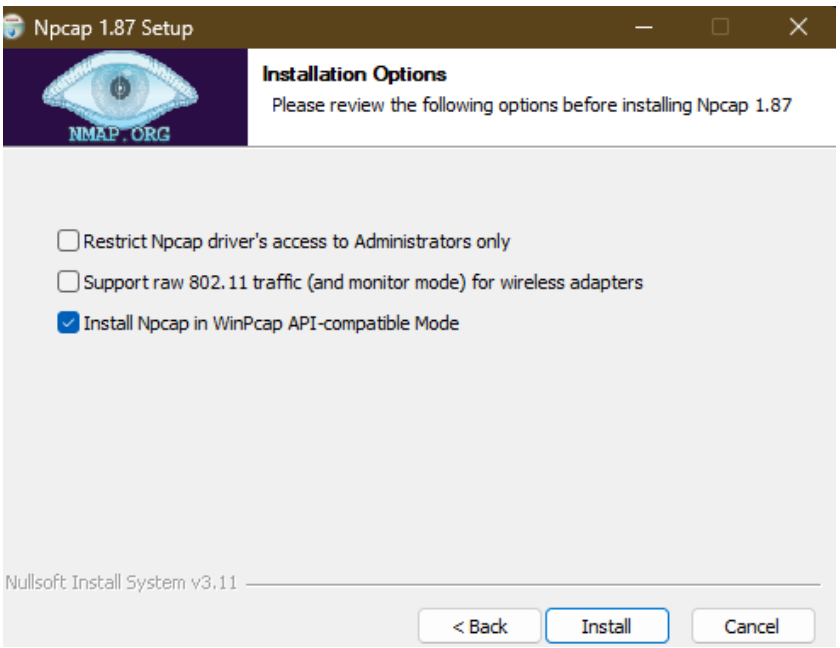


Cracked:



Ques.3> Use nmap/zenmap to analyze a remote machine.





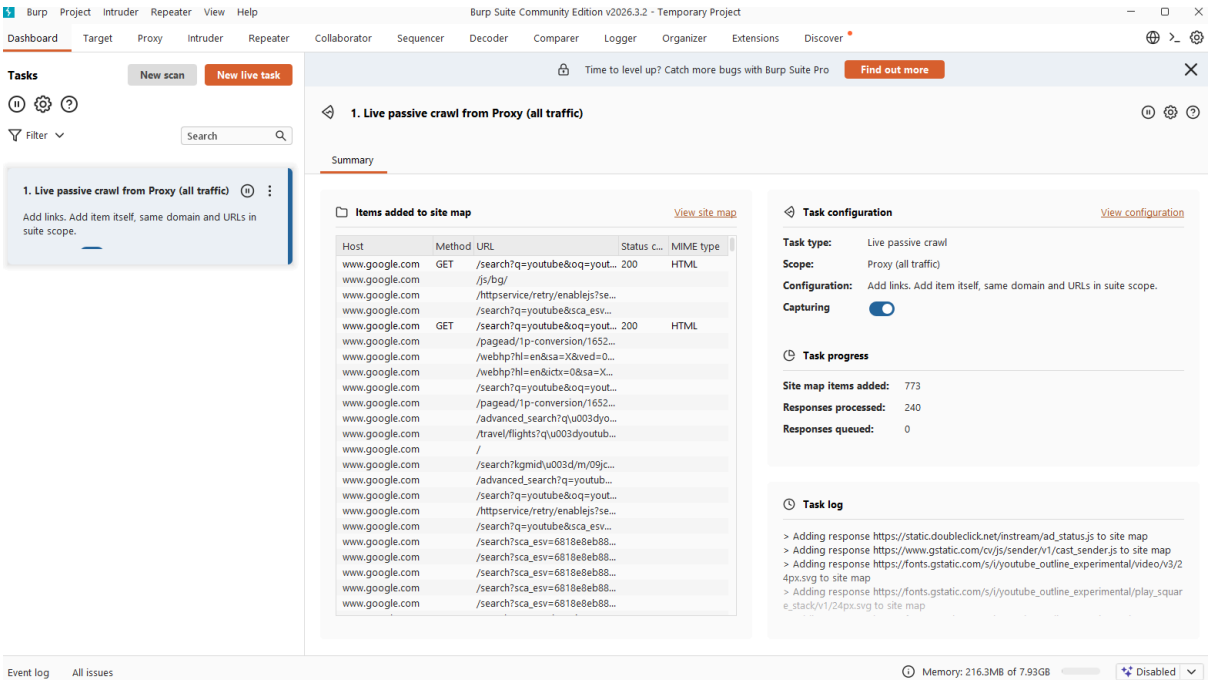

```
C:\Users\Harsh>ipconfig

Windows IP Configuration


Ethernet adapter Ethernet:

   Media State . . . . . . . . . . . : Media disconnected
   Connection-specific DNS Suffix  . : domain.name

Ethernet adapter Ethernet 2:

   Connection-specific DNS Suffix  . :
   Link-local IPv6 Address . . . . . : fe80::ffb2:3153:4b1f:9810%17
   IPv4 Address. . . . . . . . . . . : 192.168.56.1
   Subnet Mask . . . . . . . . . . . : 255.255.255.0
   Default Gateway . . . . . . . . . :

Wireless LAN adapter Local Area Connection* 3:

   Media State . . . . . . . . . . . : Media disconnected
   Connection-specific DNS Suffix  . :
```

```
C:\Users\Harsh>nmap 192.168.1.1
Starting Nmap 7.99 ( https://nmap.org ) at 2026-04-12 16:54 +0530
Nmap scan report for 192.168.1.1
Host is up (0.0054s latency).
Not shown: 997 filtered tcp ports (no-response)
PORT     STATE SERVICE
53/tcp   open  domain
80/tcp   open  http
443/tcp  open  https
MAC Address: B4:86:18:66:0E:52 (GX India Pvt)

Nmap done: 1 IP address (1 host up) scanned in 5.51 seconds

C:\Users\Harsh>
```

```
C:\Users\Harsh>nmap -sV 192.168.1.1
Starting Nmap 7.99 ( https://nmap.org ) at 2026-04-12 16:55 +0530
Nmap scan report for 192.168.1.1
Host is up (0.013s latency).
Not shown: 997 filtered tcp ports (no-response)
PORT     STATE SERVICE  VERSION
53/tcp   open  domain   dnsmasq 2.78
80/tcp   open  http     Boa HTTPd 0.94.13
443/tcp  open  ssl/http Boa HTTPd 0.94.13
MAC Address: B4:86:18:66:0E:52 (GX India Pvt)

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 26.74 seconds

C:\Users\Harsh>S
```

```
C:\Users\Harsh>nmap -O 192.168.1.1
Starting Nmap 7.99 ( https://nmap.org ) at 2026-04-12 16:59 +0530
Nmap scan report for 192.168.1.1
Host is up (0.0020s latency).
Not shown: 997 filtered tcp ports (no-response)
PORT     STATE SERVICE
53/tcp   open  domain
80/tcp   open  http
443/tcp  open  https
MAC Address: B4:86:18:66:0E:52 (GX India Pvt)
Warning: OSScan results may be unreliable because we could not find at least 1 open and 1 closed port
Aggressive OS guesses: Linux 3.10 - 4.11 (97%), Linux 3.2 - 4.14 (97%), OpenWrt Chaos Calmer 15.05 (Linux 3.18) or Desig
nated Driver (Linux 4.1 or 4.4) (91%), HP P2000 G3 NAS device (91%), Android 4 (91%), Android 5.1 (91%), Android 6 - 9 (
Linux 3.18) (91%), Linux 2.6.32 (91%), Linux 2.6.32 - 3.13 (91%), Linux 3.2 - 3.16 (91%)
No exact OS matches for host (test conditions non-ideal).
Network Distance: 1 hop

OS detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 9.36 seconds
```

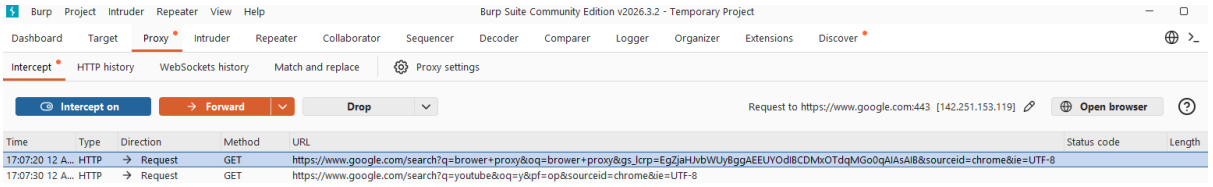
Ques4. >Use Burp proxy to capture and modify the message.



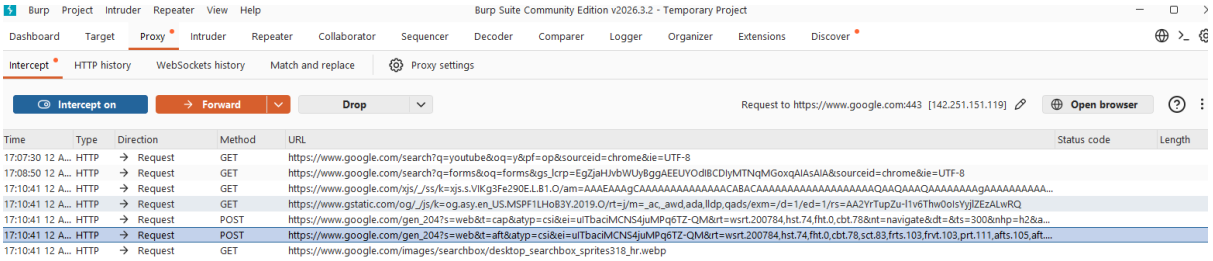
Go to:

Proxy → Intercept

Turn Intercept ON



Modify the Message and Forward the Request



Ques5. >Implement caesar cipher substitution operation.

```
text = input("Enter text: ")
shift = int(input("Enter shift: "))

result = ""
for ch in text:
    if ch.isalpha():
```

```
        base = ord('A') if ch.isupper() else ord('a')
        result += chr((ord(ch) - base + shift) % 26 + base)
    else:
        result += ch

print("Encrypted text:", result)
```

Output:

```
Enter text:  Harsh Arora
Enter shift:  4
Encrypted text: Levwl Evsve
```

Ques6. >Implement monoalphabetic and polyalphabetic cipher substitution operation.

```
import string

text = input("Enter text: ").upper()
key = input("Enter 26-letter key: ").upper()

alphabet = string.ascii_uppercase
mapping = dict(zip(alphabet, key))

cipher = ".join(mapping.get(ch, ch) for ch in text)
print("Encrypted text:", cipher)
```

```
Enter text:  Harsh Arora
Enter 26-letter key:  QWERTYUIOPASDFGHJKLZXCVBNM
Encrypted text: IQKLI QKGKQ
```

```
text = input("Enter text: ").upper()
key = input("Enter key: ").upper()

result = ""
j = 0

for ch in text:
```

```
    if ch.isalpha():
        shift = ord(key[j % len(key)]) - 65
        result += chr((ord(ch) - 65 + shift) % 26 + 65)
        j += 1
    else:
        result += ch

print("Encrypted text:", result)
```

```
Enter text:  Harsh Arora
Enter key:  CRYPTO
Encrypted text: JRPHA OTFPP
```

Ques7. > Implement playfair cipher substitution operation.

```
import numpy as np

text = input("Enter text: ").upper().replace(" ", "")
key = list(map(int, input("Enter 4 numbers (space-separated): ").split()))

if len(text) % 2 != 0:
    text += 'X'

key_matrix = np.array(key).reshape(2,2)
result = ""

for i in range(0, len(text), 2):
    pair = [ord(text[i])-65, ord(text[i+1])-65]
    res = key_matrix.dot(pair) % 26
    result += chr(res[0]+65) + chr(res[1]+65)

print("Encrypted text:", result)
```

```
Enter text:  Harsh Arora
Enter 4 numbers (space-separated):  2 4 5 8
Encrypted text: OJCVOJMPIH
```

Ques8. >Implement hill cipher substitution operation.

```
def char_to_num(c):
    return ord(c) - ord('A')

def num_to_char(n):
    return chr(n + ord('A'))

def prepare_text(text, n):
    text = text.upper()
    text = "".join(filter(str.isalpha, text))

    while len(text) % n != 0:
        text += 'X'
    return text

def hill_encrypt(text, key, n):
    text = prepare_text(text, n)
    result = ""

    for i in range(0, len(text), n):
        block = text[i:i+n]
        nums = [char_to_num(c) for c in block]

        cipher_nums = []
        for row in key:
            value = sum(row[j] * nums[j] for j in range(n)) % 26
            cipher_nums.append(value)

        result += "".join(num_to_char(num) for num in cipher_nums)

    return result
```

```
# Input matrix size
n = int(input("Enter size of key matrix (n for n x n): "))

# Input key matrix
print(f"\nEnter the {n}x{n} key matrix row by row")

key = []
for i in range(n):
    row = list(map(int, input(f"Enter row {i+1}: ").split()))

    if len(row) != n:
        print("Error: You must enter exactly", n, "values")
        exit()
    key.append(row)

# Input plaintext
plaintext = input("\nEnter plaintext: ")

cipher = hill_encrypt(plaintext, key, n)
print("\nCiphertext:", cipher)
```

```
Enter size of key matrix (n for n x n):  2

Enter the 2x2 key matrix row by row
Enter row 1:  3 4
Enter row 2:  1 6

Enter plaintext:  Harsh Arora

Ciphertext: VHTVVHDXZR
```

Ques9. >Implement rail fence cipher transposition operation.

```
def rail_fence_encrypt(text, key):
    text = text.replace(" ", "")
    rail = [[' ' for _ in range(len(text))] for _ in range(key)]
```

```
    direction_down = False
    row, col = 0, 0

    # Fill zigzag
    for char in text:
        rail[row][col] = char
        col += 1

        if row == 0 or row == key - 1:
            direction_down = not direction_down

        row += 1 if direction_down else -1

    # Print zigzag pattern
    print("\nZigzag Pattern:")
    for r in rail:
        print(" ".join(r))

    # Read row-wise
    result = ""
    for i in range(key):
        for j in range(len(text)):
            if rail[i][j] != ' ':
                result += rail[i][j]

    return result


# Input
plaintext = input("Enter plaintext: ")
key = int(input("Enter number of rails: "))

cipher = rail_fence_encrypt(plaintext, key)
```

```
print("\nCiphertext:", cipher)
```

```
Enter plaintext:  Harsh Arora
Enter number of rails:  4

Zigzag Pattern:
H           r
  a       A   o
    r   h       r
      s           a

Ciphertext: HraAorhrsa
```

Ques10. > Implement row transposition cipher transposition operation.

```
import math
import random


def generate_key(n):
    key = list(range(1, n+1))
    random.shuffle(key)
    return key


def row_transposition_encrypt(text, key):
    text = text.replace(" ", "")
    n_cols = len(key)
    n_rows = math.ceil(len(text) / n_cols)

    # Fill matrix row-wise
    matrix = []
    index = 0

    for i in range(n_rows):
        row = []
        for j in range(n_cols):
            if index < len(text):
                row.append(text[index])
                index += 1
            else:
                row.append('X')
```

```
        matrix.append(row)

    # Print matrix
    print("\nMatrix:")
    for r in matrix:
        print(" ".join(r))

    # Column order based on key
    order = sorted(list(enumerate(key)), key=lambda x: x[1])

    # Read columns
    cipher = ""
    for idx, _ in order:
        for row in matrix:
            cipher += row[idx]

    return cipher


# Input
plaintext = input("Enter plaintext: ")
n_cols = int(input("Enter number of columns: "))

key = generate_key(n_cols)
print("Generated Key:", key)

cipher = row_transposition_encrypt(plaintext, key)
print("\nCiphertext:", cipher)
```

```
Enter plaintext:  Harsh Arora
Enter number of columns:  3
Generated Key: [2, 3, 1]

Matrix:
H a r
s h A
r o r
a X X

Ciphertext: rArXHsraahoX
```

Ques11. >Implement product cipher transposition operation

```
import math
import random

def generate_key(n):
    key = list(range(n))
    random.shuffle(key)
    return key

def transposition(text, cols, key):
    rows = math.ceil(len(text) / cols)

    # Fill matrix
    matrix = []
    index = 0

    for i in range(rows):
        row = []
        for j in range(cols):
            if index < len(text):
                row.append(text[index])
                index += 1
            else:
                row.append('X')
```

```
        matrix.append(row)

    # Print matrix
    print("\nMatrix:")
    for r in matrix:
        print(" ".join(r))

    # Read using key order
    cipher = ""
    for k in sorted(range(len(key)), key=lambda x: key[x]):
        for r in range(rows):
            cipher += matrix[r][k]

    return cipher


# Input
plaintext = input("Enter plaintext: ").replace(" ", "")
cols = int(input("Enter number of columns: "))

# First key
key1 = generate_key(cols)
print("Key 1:", key1)

# First transposition
step1 = transposition(plaintext, cols, key1)
print("\nAfter First Transposition:", step1)

# Second key
key2 = generate_key(cols)
print("\nKey 2:", key2)

# Second transposition
final_cipher = transposition(step1, cols, key2)
```

print("\nFinal Ciphertext:", final_cipher)

```
Enter plaintext:  Harsh Arora
Enter number of columns:  3
Key 1: [0, 2, 1]

Matrix:
H a r
s h A
r o r
a X X

After First Transposition: HsrarArXahoX

Key 2: [2, 0, 1]

Matrix:
H s r
a r A
r X a
h o X

Final Ciphertext: srXorAaXHarh
```